Additional Lean Proof Simplifications

After the prior proposal corpus

Three proof-only integration candidates and one conditional structural direction

Prepared for Vladimir Reshetnikov 26 October 2026

Main result. Three small, source-grounded candidates remain after treating the earlier proposals as available. They remove **23 physical lines** across three complete modules, including every added production helper and import. They preserve the six affected theorem statements and the executable definitions. A fourth idea gives a common proof interface for numeric control overwrites, but its infrastructure is not yet justified by measured savings.

Validation boundary. Four standalone files compile with Lean 4.32.0. They check the central mathematical arguments and exact proof patterns, with empty axiom sets or `propext` only. All three patches apply cleanly at the source pin. The Mathlib-dependent production modules have **not** been compiled in this review. The distinction is maintained throughout the packet.

Non-repetition boundary. The comparison covers 23 placed proposal packets and five incoming archives through `e429ae09c367`. It includes optional, deferred, and uncompiled proposals. The measured source pin is `89226e3921bc`; the three patch targets and the control-word target are unchanged at the freshness pin. These are additional applications in this repository, not claims of new mathematics or new Lean library techniques.

Candidate module	Before	After	Saved	Production cost added
`CurrentSelectorTail2011`	335	322	13	None
`CurrentControllerData2011`	184	178	6	One helper; one import
`CurrentTileColors2011`	104	100	4	None
Total	**623**	**600**	**23**	Included above

The numbers are physical-line measurements of the supplied source candidates. They are neither a speed measurement nor a count for a hypothetical checkout containing every alternative design in the old reports. The full diffs, candidate modules, source identities, comparison inventories, and reproducible checks accompany this article.

Reading guide. Sections 2–4 explain the three concrete patches. Section 5 develops the conditional control-slot idea. Section 6 separates the completed checks from the production integration gate, and Section 7 gives specific further questions. The archive README maps every supporting artifact and its replay command.

1 Scope, source state, and the exclusion standard

The requested tree is unusually well covered by its existing proposal corpus. The obvious large refactors are already recorded: shared residual-square arguments, generator-backed circuit proofs, scanner iteration laws, coefficient envelopes, decoder invariants, internal arithmetic interfaces, cyclic residue maps, carry uniqueness, Pell growth kernels, canonical helical coordinates, and constructed Wang rows. Presenting another version of any of these as a new recommendation would not satisfy the request.

The initial review therefore indexed the complete placed collection, `2026-10-26a` through `2026-10-26w`, and the actual incoming archives, rather than relying on the incoming README's queue description. At the initial pin there were 1,286 files under the placed collection and four ZIPs containing 911 non-directory members. The report and supplementary-text index contains 2,028 heading occurrences. These counts describe artifacts and repeated discussion; they are not counts of independent ideas or verified theorems.

A detailed refresh found twelve additional commits through `663fb267f614`, including a fifth incoming archive with 131 members. The final comparison corpus therefore contains **28 packets**: 23 placed packets and five incoming archives, with 1,042 incoming members in total. The new archive was read and checked against all four retained ideas. Its topics include periodic helices, mixed-sum divisibility, geometric-series bounds, quadratic margins, signed prefix borrow arithmetic, and scalar rank comparisons. None proposes the four changes developed here. A final tree comparison through `e429ae09c367`found three further commits with no changes in either proposal root or any of the four target files. The freshness records include both comparisons.

The source measurements use the full commit

`89226e3921bce3e8949fadd12d3efb6914c7630c`.

The closing comparison uses

`e429ae09c367c28602613d1d7351a96968aa2f93`.

The target files are unchanged between these commits. Other recent commits adopted earlier coefficient, decoder, evaluator, carrier, and option-map refactors. Those changes are treated as existing work and are not counted here.

1.1 What "assume the earlier ideas applied" means

The earlier packets contain mutually alternative designs as well as concrete patches. There is no single meaningful operation that mechanically applies every packet into one compiling checkout. The comparison is instead at the level of the mathematical contract and the proof burden it removes. For each new candidate, this report identifies the closest earlier contract and asks what work remains after that contract is available.

For example, an earlier binary-word congruence interface already gives the word equality used by the color proof. The additional candidate addresses only the subsequent cancellation of bounded natural complements. Likewise, an earlier physical-permutation injectivity proposal is assumed available; the tail candidate addresses two clients of an already established strict-order equivalence. This prevents the same abstraction from being counted again under a new theorem name.

Several apparently attractive candidates were discarded for this reason. A residue ring homomorphism from $\mathbb{Z}/(NL)\mathbb{Z}$ to $\mathbb{Z}/L\mathbb{Z}$, including its representative formula, is already explicit in packet p. The marker and clause applications already appear in packet a. Shared parity extraction from carry balance is also

old, and packet o explicitly withdraws that proposal as repetition. The origin-mask encode/decode direction largely follows from the earlier no-borrow identity and word canonicality. None is included as a new direction in this article.

2 Derive the selector-tail cutoff by logical duality

2.1 Existing proof burden

The relevant source is `CurrentSelectorTail2011.lean`. Its physical positions are sorted in decreasing selector-weight order, with Start at the exceptional position zero. Comparing a non-Start tile i to the fixed cutoff tile 866 gives an existing theorem of the form

$$p(i) < 459 \quad \longleftrightarrow \quad q < w(i),$$

where p is the physical position, w is the selector weight, and q is the cutoff weight. The source also already proves

$$w(i) \le q \quad \longleftrightarrow \quad \mathrm{south}(i) \le 45.$$

The desired suffix classifier is

$$459 \le p(i) \quad \longleftrightarrow \quad i \neq 0 \ \wedge \ \mathrm{south}(i) \le 45.$$

After handling Start, the existing proof manually builds both directions. Each direction assumes the opposite strict inequality and obtains a contradiction using the strict-order iff. This is precisely the content of negating that iff. No new ranking theorem or finite-table fact is needed.

2.2 Replacement and its proof

For natural numbers, negating $p < c \leftrightarrow q < w$ gives $c \le p \leftrightarrow w \le q$. This is an ordinary consequence of linear order, and Lean can express it directly:

```
theorem cutoff_duality (p c q w : Nat)
    (h : p < c ↔ q < w) : c ≤ p ↔ w ≤ q := by
  simpa only [Nat.not_lt] using not_congr h
```

This standalone theorem is explanatory evidence, not a new production helper. In the production proof, retain the exceptional Start branch and the two existing lines which construct and normalize `hr`. Replace the handwritten implication pair by:

```
    rw [and_iff_right hi, ← weight_le_cutoff_iff]
    simpa only [Nat.not_lt] using not_congr hr
```

The guard `hi` is still $i \neq 0$. The rewrite removes the redundant conjunct under that hypothesis. The second rewrite changes the South-bound predicate back to the weight cutoff, so the goal is exactly the negation of `hr`. This proof depends on the same ordering and cutoff facts as before.

There is a second payoff in `selectorTail_succ`. Its two branches merely transport an already-proved equivalence into an `if`. The nonzero fact for a finite successor is a library theorem. The complete replacement body is:

```
simp only [selectorTailTerm, tail_position_iff, ne_eq, Fin.succ_ne_zero,
  not_false_eq_true, true_and, lowMass, mul_ite, mul_zero]
```

Here `mul_ite` moves multiplication by two through the conditional. The `ne_eq` rewrite is intentional: with this restricted simplifier set, the standalone check found that omitting it left the nonzero guard unresolved. The final patch includes it.

2.3 Incremental benefit and preserved boundaries

The declaration for `tail_position_iff` changes from 15 to 10 physical lines, and `selectorTail_succ` from 12 to 4. The complete module changes from 335 to 322 lines. No helper, import, data table, or rank definition is added. The literal cutoff position 459, cutoff tile 866, South threshold 45, exceptional Start slot, exact mass calculations, and coefficient bounds all remain as they were.

Packet m's physical-permutation proposal addresses injectivity. The incoming signal-mass proposal addresses how counts and weighted masses are derived. Neither removes these two logical conversions. The candidate is therefore a small additional application after those proposals, rather than a replacement for either of them.

The standalone `TailCutoffCore.lean` checks the exact non-Start proof, the exceptional Start branch with `Fin 942`, and the successor conditional normalization. The upstream table and strict-order results are explicit hypotheses in that file. It checks how those facts are used; it does not re-prove their repository implementations. The production candidate remains subject to the normal imported-module check.

3 Replace prime-divisor case trees with finite order certificates

3.1 The mathematical criterion already exists

The source `CurrentControllerData2011.lean` proves that 2 has order 23 modulo 47 and order 402 modulo 2011. It already uses Mathlib's `orderOf_eq_of_pow_and_pow_div_prime`. The cumbersome part is the proof that a prime dividing 402 must be one of 2, 3, or 67. The current source repeatedly splits divisibility of products and converts prime divisibility into equality.

For an element x of a monoid, a positive candidate order n is exact if

$$x^n = 1, \qquad \forall p \in \text{primeFactors}(n), \quad x^{n/p} \neq 1.$$

The mathematical reason is that $x^n = 1$ makes the actual order divide n. If the actual order were a proper divisor, some prime divisor of the quotient would yield a smaller exponent n/p at which x is already 1. Mathlib supplies the criterion; this proposal changes the finite interface for its remaining premise.

The exact membership theorem in the pinned Mathlib version is

$$p \in n.\texttt{primeFactors} \quad \longleftrightarrow \quad \text{Prime}(p) \wedge p \mid n \wedge n \neq 0.$$

Thus the finite certificate is complete, not just a list of convenient tests. This distinction matters: listing a few proper powers without proving that the list contains every relevant prime divisor would not establish the order.

3.2 A small adapter with an accounted cost

The candidate adds `Mathlib.Data.Nat.PrimeFin` and one private theorem:

```
private theorem order_eq_of_factors {M : Type*} [Monoid M] (x : M) {n : ℕ}
    (hn : 0 < n) (hpow : x^n = 1)
    (hproper : ∀ p ∈ n.primeFactors, x^(n/p) ≠ 1) : orderOf x = n := by
  apply orderOf_eq_of_pow_and_pow_div_prime hn hpow
  intro p hp hpn
  exact hproper p (Nat.mem_primeFactors.mpr ⟨hp,hpn,Nat.ne_of_gt hn⟩)
```

The two public clients then reuse their existing power-equals-one facts:

```
theorem two_order_47 : orderOf (2 : ZMod 47) = 23 :=
  order_eq_of_factors _ (by decide) two_pow_47 (by decide)

theorem two_order_2011 : orderOf (2 : ZMod 2011) = 402 :=
  order_eq_of_factors _ (by decide) two_pow_2011 (by decide)
```

The final calls use ordinary `decide` on closed finite propositions. Their concrete Mathlib elaboration and reduction cost still need to be measured. An earlier prototype that explicitly rewrote each prime-factor set and then split its members did not repay the helper cost as well. The delivered patch uses the closed certificate directly and includes its full cost in the measurement.

The complete module changes from 184 to 178 physical lines: 18 deleted and 12 added, including the import and the six-line private theorem. Existing named proper-power residues remain in the module, as do the order facts for the large base, the auxiliary lift, and the conditional kernel-residue wrappers. There is no new claim that the actual kernel residues have been discharged.

3.3 Guards and edge cases

The premise $n > 0$ must remain. At $n = 0$, the prime-factor set is empty and $x^0 = 1$ holds automatically. Removing the positivity guard would therefore turn an empty finite check into a false general order assertion. At $n = 1$, the prime-factor check is correctly empty: $x^1 = 1$ itself proves that the order is one.

No primality of the ambient modulus is used by the adapter. The finite tests concern prime divisors of the *candidate order*, not the modulus. This is why the adapter is stated for a monoid, with no finite-group instance. The two particular moduli happen to be prime, but that fact is not smuggled into the criterion as an unnecessary assumption.

The nearest prior finite-certificate proposals concern tile tables, chunks, coefficient fibres, or decoder bounds. This candidate concerns the completeness of the exact-order test family. It is a new local application of an existing order theorem, with a small measured payoff, not a new certificate engine.

3.4 Independent kernel evidence for the concrete numbers

The supplied `FiniteOrderCore.lean` uses a small recursive evaluator that reduces modulo m after each multiplication:

$$r_0 = 1 \bmod m, \qquad r_{k+1} = r_k b \bmod m.$$

It proves $r_k = b^k \bmod m$ for every b, m, k, then checks the positive return time and *every* smaller positive exponent for the two candidates. Consequently it establishes the ordinary arithmetic statements

$$\begin{aligned} 2^{23} &\equiv 1 \pmod{47}, \qquad 0 < k < 23 \Longrightarrow 2^k \not\equiv 1 \pmod{47}, \\ 2^{402} &\equiv 1 \pmod{2011}, \quad 0 < k < 402 \Longrightarrow 2^k \not\equiv 1 \pmod{2011}. \end{aligned}$$

It also checks the three residues relevant to the smaller factor certificate:

$$2^{201} \bmod 2011 = 2010, \quad 2^{134} \bmod 2011 = 205, \quad 2^6 \bmod 2011 = 64.$$

The finite certificate declarations and the three residue computations have empty axiom sets. The evaluator correctness and the resulting ordinary first-return theorems use only `propext`. This evidence rules out an incorrect numerical order or missing smaller return. It does *not* certify the imported `primeFactors` adapter, its typeclass elaboration, or the proposed production `decide` calls. Those remain a distinct gate.

4 Cancel bounded color complements directly

4.1 The remaining step after word congruence

The source `CurrentTileColors2011.lean` first derives equality of the binary words carrying adjacent labels. Each word is known to be the complement of a tile color. For horizontal colors the resulting equality has the form

$$255 - \mathrm{east}(i) = 255 - \mathrm{west}(j),$$

and for temporal colors it is

$$511 - \mathrm{south}(i) = 511 - \mathrm{north}(j).$$

The current proofs unpack all four color bounds for each of two tiles, rewrite the word equalities, and let `omega` finish. Only two of the eight bounds are needed in either proof.

The relevant existing Mathlib result is `tsub_right_inj`:

$$b \le a \ \wedge \ c \le a \quad \Longrightarrow \quad (a - b = a - c \ \longleftrightarrow \ b = c).$$

The candidate applies it using just the needed projections of `tile_bounded`. For example, the horizontal final step becomes:

```
exact (tsub_right_inj (Nat.le_of_lt_succ (tile_bounded i).2.1)
  (Nat.le_of_lt_succ (tile_bounded j).1)).mp
    (hl.2.2.1.symm.trans (hw.trans hr.1))
```

The displayed version is wrapped for readability; the candidate file uses two physical lines. The equality chain first moves from the left color complement to its word, then across the already-proved word equality, then to the right color complement. The temporal proof uses the South and North bound projections in exactly the same manner.

4.2 Why the bounds are essential

Natural subtraction truncates at zero. Thus equality of two unbounded complements does not imply equality of the underlying numbers. The standalone check includes the counterexample

$$255 - 256 = 255 - 257 = 0, \qquad 256 \neq 257.$$

The proof therefore retains both bounds explicitly. Given $a, b \leq c$, apply $v \mapsto c - v$ to $c - a = c - b$, then use $c-(c-a) = a$ and $c-(c-b) = b$. This exact argument is compiled in `BoundedComplementCore.lean`.

The complete module changes from 104 to 100 physical lines, with no helper or import added. The two final four-line proof blocks become two lines each. All decoded-word equalities, copy orientation, temporal shift information, and tile bounds remain in use. The core theorem's axiom set is `[propext]`; the production module is uncompiled in this review.

This is deliberately the lowest-priority patch. It makes the cancellation principle explicit and avoids eight named inequalities, but the nested projection and equality-chain notation may be less pleasant than `omega` to some maintainers. Its case rests on making the two required guards visible, not on a four-line count alone. Keeping the original proof is reasonable if the local style favors the original arithmetic conclusion.

Earlier proposals already supply binary-word congruence and color-fibre interfaces. Those give the equality consumed here. The candidate addresses only bounded-complement recovery, so it remains a distinct, modest residual cleanup after those interfaces are assumed available.

5 Conditional direction: numeric control-slot invariants

5.1 An exact proof-side view of the current word

The source `CurrentWordConstruction2011.lean` assembles a cell from a tile's local digit function and two independent numeric controls. At field 977 the value is the dummy control; at field 1004 it is the marker control; elsewhere it is the local tile digit. The same function can be viewed on the proof side as

$$f[1004 \mapsto \text{marker}][977 \mapsto \text{dummy}].$$

The order is deliberate: the final update has the same priority as the first branch of the original nested conditional. The actual two positions differ, but the general update law should also be correct for repeated positions.

Mathlib already supplies `Function.forall_update_iff`. In the nondependent case it expresses exactly the invariant rule

$$\big(\forall j,\ P(j, f(j))\big) \ \wedge\ P(i, a) \quad \Longrightarrow \quad \forall j,\ P\big(j, f[i \mapsto a](j)\big).$$

The update view is a proof interface. There is no reason to change the public executable definition of `word` to adopt the argument.

5.2 Three distinct invariant instances

Boolean range uses $P(e, v) \equiv v \leq 1$. The base certificate comes from `localDigit_boolean`; each overwrite requires its own Boolean bound. This reproduces the intended independence of the dummy and marker maps.

Support preservation uses

$$P(e, v) \equiv (e \notin \text{lowSupport} \implies v = 0).$$

Since both control positions belong to `lowSupport`, the overwritten values can be *arbitrary natural numbers*. There is no Boolean premise in the original `word_support` theorem, and none may be added by this refactor. A combined invariant such as Booleanity plus support is useful for some clients, but cannot replace the stronger support theorem by a weaker statement with extra assumptions.

Capacity preservation uses $P(e, v) \equiv v < \text{capacity}(e)$. Its update obligations are the two local capacity inequalities. This gives a possible common interface if future assembled-word proofs repeatedly need Boolean, support, and varying bit-capacity predicates. It does not eliminate the separate arithmetic facts needed to establish those capacities.

5.3 Evidence, novelty, and the reason for deferring adoption

`ControlSlotCore.lean` checks the general overwrite law and the three invariant instances with Lean 4.32.0. It also checks both actual control positions, an unaffected position, and repeated-position last-write priority. The four audited theorems use only `propext`. The core representation uses an explicit conditional and has a definitional equality to the literal two-branch expression.

`ControlSlotInvariant.lean` supplies the proposed Mathlib update view and its binding to the actual production word. That binding is uncompiled. The nearest earlier proposal, locality of expression evaluation under agreement of valuations, solves a different problem: it transports an evaluator between environments. Here the invariant belongs to the numeric digit function being overwritten. The narrow application was not found in the comparison corpus.

Nevertheless, the full update facade adds definitions, a binding theorem, and an invariant adapter. For only the current Boolean and support clients, that cost has not been shown to pay back. It would be misleading to count a short projection proof without charging the facade that makes it possible. The prototype is therefore supplied as a conditional direction, excluded from the 23-line total and from the combined patch.

The sensible adoption test is concrete: implement all intended clients, preserve the unconditional support statement, keep the executable word unchanged, and measure the entire production diff. If the total increases without a substantial reduction in repeated proof obligations, retain the current local proofs. A new abstraction is optional even when its law is correct and reusable.

6 Validation, accounting, and integration

6.1 What was actually checked

The compiler is the repository-pinned Lean 4.32.0 release. The delivered `verify_core.py` checks its version, compiles the four standalone files, records their SHA-256 identities and actual exit codes, and retains complete compiler output. The successful final logs contain no `sorryAx`. Across the four files there are 16 printed axiom reports; each is empty or contains only `propext`. Compilation time in these logs is evidence about those standalone checks only.

The supplied `check_candidates.py` performs different, textual checks: it compares the pinned source bytes to the local checkout, compares all six edited statement headers, counts the complete-module changes, and runs `git apply -check` on each patch and on their combination. These checks passed.

They establish that the artifacts are internally consistent and applicable at the pin; they do not establish Lean elaboration.

Evidence	What it establishes
Tail core compilation	Logical duality, exceptional Start branch, and conditional normalization under stated upstream hypotheses.
Finite-order core compilation	Ordinary modular first-return statements, all smaller positive exponents, and the three proper-power residues.
Complement core compilation	Bounded natural-complement cancellation and a counterexample without the guards.
Control-slot core compilation	General overwrite invariant, Boolean/support/capacity instances, and update priority.
Patch and header checks	Applicable textual edits, measured line counts, and unchanged six theorem headers.

The complete Mathlib-dependent import graph was not available for a production replay. Therefore no changed production module, downstream client, concrete update binding, or production axiom closure is reported as compiled here. There is also no claim of a compile-time speedup. In particular, the concise factor certificates could elaborate differently from the explicit `reduce_mod_char` steps they replace.

6.2 Recommended order of adoption

The selector-tail patch has the clearest balance: it adds no infrastructure, retains the full exceptional case, and removes a duplicated logical conversion. Check it first, with its existing check module and the early padding consumer.

The order-certificate patch should come next. Check the two closed finite certificates in the real Mathlib environment, retain the existing explicit proper-power facts, and compare both axiom reports and actual elaboration cost. If `decide` performs poorly, the same factor-set interface can enumerate the three modular exclusions explicitly, but the source cost must then be recalculated. The supplied six-line saving is attached to the exact delivered candidate, not to every implementation of the idea.

The bounded-complement patch is a style-sensitive final cleanup. Its normal integration gate is the color check module and the marked-window consumer. The overwrite facade should be considered only after a full set of concrete clients justifies it.

In a disposable pinned checkout, apply the small diffs and build the changed modules, their existing checks, and the affected direct consumers. The exact command is in the README. Compare the public axiom reports with the original checkout. The full candidate files are reference artifacts; if other proposals have reorganized the same modules, rebase these proof bodies instead of overwriting the updated files with the older snapshots.

6.3 Scope of the proof edits

These edits concern proof presentation and reuse, with the local statements and their hypotheses preserved. Results added by the surrounding research program must be assessed from their own evidence. The present packet supplies no additional mathematical result about the global controller, equation construction, or consistency equivalence.

7 Further questions with specific acceptance criteria

Does a shared exact-order certificate earn its cost across more controller constants? The current pair pays for the private adapter by six physical lines. Further clients could improve that balance, but should be counted only after the constants, factor-set completeness, and proper powers are explicit. Compare actual checking cost with the existing modular exponentiation tactic; do not infer speed from shorter source.

Can the numeric-control invariant replace three independently maintained families of obligations? A useful experiment would cover support, Boolean range, and bit capacities together while keeping the unrestricted support theorem. Its acceptance condition is a complete diff with either a net reduction or an independently defensible maintenance benefit. A short client built on a large uncounted facade is insufficient.

Are there more non-strict classifiers already implied by strict order comparisons? Search specifically for pairs of handwritten contraposition branches after an existing iff. Each candidate should retain its exceptional elements, ties, and boundary conventions. This is a focused search for new clients, not another proposal for a global rank or order API.

When should explicit cancellation replace arithmetic automation? The tile-color example suggests a useful local criterion: use the library lemma when it makes the exact two guards easier to see. Keep automation when the explicit projection chain obscures the statement. A maintainable result need not be the version with the fewest printed lines.

A Primary sources

All repository claims use pinned source rather than the moving main branch. The article's new contribution is the proposed simplification and its stated reasoning. The following links support the inspected source and library APIs.

1. Placed proposal corpus, including its index and adoption record.
2. Incoming archives at the closing comparison pin.
3. Selector-tail source; existing selector-tail checks.
4. Controller arithmetic source; existing controller checks.
5. Tile-color source; existing color checks.
6. Assembled-word source; capacity and low-mask clients.
7. Mathlib v4.32.0, PrimeFin: exact prime-factor membership interface.
8. Mathlib v4.32.0, OrderOfElement: exact-order criterion with positive candidate order.
9. Mathlib v4.32.0, ordered subtraction: `tsub_right_inj`.
10. Mathlib v4.32.0, Function.Basic: update application and pointwise invariant interface.
11. Lean v4.32.0 finite-index lemmas: `Fin.succ_ne_zero`.